

TP5 — Multi-Projets : Pipelines Parents-Enfants et Includes (version debutant)

Objectifs

A la fin de ce TP, vous saurez :

1. Ce que sont les **pipelines multi-projets** et pourquoi ils existent
2. Utiliser les 4 types d'**include** (local, file, template, remote)
3. Créer des **templates CI partages** réutilisables entre projets
4. Déclencher un **pipeline enfant** (child pipeline) depuis un pipeline parent
5. Déclencher un **pipeline downstream** dans un autre projet GitLab
6. **Passer des variables** entre pipelines parents et enfants
7. Organiser un projet GitLab en **plusieurs fichiers CI**

Prerequis

- Avoir complété les TP1 à TP4
- Un projet GitLab avec un pipeline fonctionnel
- Savoir faire `git add`, `git commit`, `git push`
- Comprendre les stages, les jobs, le cache et les artifacts

Etape 1 — Pourquoi les pipelines multi-projets ?

Le probleme

Imaginez une entreprise qui a 3 projets :

```
mon-groupe/  
  api-backend/      --> Application Express (Node.js)  
  lib-common/       --> Librairie partagée (fonctions utilitaires)  
  app-frontend/     --> Application React
```

Chaque projet a son propre `.gitlab-ci.yml`. Mais ils partagent des besoins communs :

- Installer Node.js de la même manière
- Utiliser le même cache npm
- Lancer les mêmes commandes de base (`npm ci`, `npm test`, `npm run build`)
- Quand l'API est déployée, il faut aussi déployer le frontend

Sans multi-projets, vous devez :

- Copier-coller la même configuration dans chaque projet (violation du DRY)

- Deployer manuellement le frontend apres l'API
- Maintenir 3 fichiers identiques a 80%

L'analogie

Pensez a une **chaîne de montage automobile** :

SANS multi-projets (artisanal) :

- Usine Moteur : fabrique le moteur, fait ses propres tests
- Usine Carrosserie : fabrique la carrosserie, fait ses propres tests
- Usine Assemblage : assemble tout... mais ne sait pas si le moteur est pret !

AVEC multi-projets (industriel) :

- Usine Moteur : fabrique + teste, puis NOTIFIE l'usine d'assemblage
- Usine Carrosserie : fabrique + teste, puis NOTIFIE l'usine d'assemblage
- Usine Assemblage : attend les notifications, puis assemble avec confiance

Les deux mecanismes GitLab

Mecanisme	Ce que c'est	Quand l'utiliser
include	Importer des morceaux de YAML dans votre pipeline	Reutiliser des templates, factoriser la config
trigger	Lancer un autre pipeline (enfant ou autre projet)	Orchestrer plusieurs projets ensemble

Etape 2 — Comprendre les 4 types d'include

Le mot-cle `include` permet d'importer du YAML depuis differentes sources.

2.1 — `include:local`

Importe un fichier **du meme depot** (meme projet Git).

```
include:
  - local: '.gitlab/ci/templates.yml'
  - local: '.gitlab/ci/deploy.yml'
```

****Quand l'utiliser ?**** Pour decouper un gros `.gitlab-ci.yml` en plusieurs fichiers dans le meme projet. C'est

2.2 — `include:file`

Importe un fichier depuis **un autre projet GitLab** (sur la meme instance).

```
include:
  - project: 'mon-groupe/ci-templates'
    ref: main
    file: '/templates/node-base.yml'
```

****Quand l'utiliser ?**** Pour centraliser les templates CI dans un projet dedie et les partager entre tous vos p

2.3 — include:template

Importe un **template officiel GitLab** (maintenu par GitLab).

```
include:
  - template: 'Security/SAST.gitlab-ci.yml'
  - template: 'Jobs/Code-Quality.gitlab-ci.yml'
```

****Quand l'utiliser ?**** Pour ajouter rapidement des fonctionnalités standard (analyse de sécurité, qualité de c

2.4 — include:remote

Importe un fichier depuis **n'importe quelle URL** accessible.

```
include:
  - remote: 'https://example.com/ci/shared-config.yml'
```

****Quand l'utiliser ?**** Pour importer des templates publics depuis GitHub ou un serveur web. Attention à la sec

Tableau récapitulatif des includes

Type	Source	Exemple d'usage
local	Même dépôt	Découper son pipeline en fichiers
file	Autre projet GitLab	Templates partagés dans un groupe
template	Templates officiels GitLab	SAST, Code Quality, Auto DevOps
remote	URL publique	Template open-source sur GitHub

Étape 3 — Créer le projet sur GitLab

1. Allez sur <https://gitlab.com>
2. **New project > Create blank project**
3. **Project name** : tp5-multi-projets
4. **Décochez** **"Initialize repository with a README"**
5. Cliquez sur **Create project**

Étape 4 — Préparer le code du projet

```
# Aller dans le dossier du jour 3
cd ~/Desktop/"CI CD"/jour3

# Creer un dossier de travail propre
mkdir tp5-work
cd tp5-work

# Copier les fichiers du projet-demo
cp -r ../projet-demo/src ./
cp -r ../projet-demo/tests ./
cp -r ../projet-demo/scripts ./
cp ../projet-demo/package.json ./
cp ../projet-demo/.gitignore ./

# Copier TOUS les fichiers CI du TP5
cp ../tp5-multi-projets/.gitlab-ci.yml ./
mkdir -p .gitlab/ci
cp ../tp5-multi-projets/.gitlab/ci/templates.yml ../.gitlab/ci/
cp ../tp5-multi-projets/.gitlab/ci/deploy.yml ../.gitlab/ci/
cp ../tp5-multi-projets/child-pipeline.yml ./

# Verifier la structure
find . -name "*.yml" -not -path "./node_modules/*"
```

Vous devez voir :

```
./.gitlab-ci.yml
./.gitlab/ci/templates.yml
./.gitlab/ci/deploy.yml
./child-pipeline.yml
```

Tester en local :

```
npm install
npm test
```

Vous devez voir Tests: 17 passed, 17 total. Puis :

```
rm -rf node_modules
```

Etape 5 — Comprendre le template partage (templates.yml)

Ouvrez le fichier .gitlab/ci/templates.yml dans VS Code :

```
code .gitlab/ci/templates.yml
```

Ce fichier definit un job "cache" reutilisable

```
.base-node:
  image: node:18-alpine
  cache:
    key:
      files:
        - package-lock.json
    paths:
      - node_modules/
      - .npm/
  policy: pull-push
  before_script:
    - 'echo "Etape : installation des dependances"'
    - npm ci --cache .npm --prefer-offline
```

****Le point important : le `` devant le nom****

Un job dont le nom commence par `` est un ****job cache**** (hidden job). Il ne s'exécute ****jamais**** directement.

C'est exactement comme une classe abstraite en programmation : on ne l'instancie pas, on en hérite.

Comment l'utiliser dans un autre fichier ?

```
test-unit:
  extends: .base-node
  stage: test
  script:
    - npm test
```

Le job test-unit **hérite** de tout ce qui est dans .base-node :

- L'image node:18-alpine
- La configuration du cache
- Le before_script qui installe les dépendances

Il n'a plus qu'à définir son propre script.

Etape 6 — Comprendre le pipeline principal (.gitlab-ci.yml)

Ouvrez le fichier .gitlab-ci.yml dans VS Code :

```
code .gitlab-ci.yml
```

Structure du pipeline

Le pipeline principal fait 3 choses :

1. **Inclut** les templates et le fichier de deploy
2. **Exécute** les jobs classiques (install, test, build)
3. **Declenche** un pipeline enfant

Les includes

```
include:
  - local: '.gitlab/ci/templates.yml'
  - local: '.gitlab/ci/deploy.yml'
```

Ces deux lignes importent le contenu de `templates.yml` et `deploy.yml` dans le pipeline. C'est comme si vous aviez tout copié-collé dans un seul fichier, mais en **beaucoup plus propre**.

Le job trigger (pipeline enfant)

```
trigger-child:
  stage: trigger
  trigger:
    include: child-pipeline.yml
    strategy: depend
  variables:
    PARENT_VERSION: $APP_VERSION
    PARENT_PIPELINE_ID: $CI_PIPELINE_ID
```

****Que fait ce job ?****

Il crée un **nouveau pipeline** (le pipeline enfant) à partir du fichier `child-pipeline.yml`. Ce pipeline enf

****strategy: depend**** signifie que le pipeline parent **attend** que le pipeline enfant soit terminé avant de

Visualisation dans GitLab

```
Pipeline parent (.gitlab-ci.yml)
|
+-- install-deps      [install]
+-- test-unit         [test]
+-- build-app         [build]
+-- trigger-child     [trigger] --> Pipeline enfant (child-pipeline.yml)
|
|                                     |
|                                     +-- validate-deployment [validate]
|                                     +-- notify-team           [notify]
|
+-- deploy-staging    [deploy] (depuis deploy.yml)
+-- deploy-production [deploy] (depuis deploy.yml, manuel)
```

Etape 7 — Comprendre le pipeline enfant (child-pipeline.yml)

Ouvrez le fichier `child-pipeline.yml` dans VS Code :

```
code child-pipeline.yml
```

Ce pipeline reçoit des variables du parent

```
variables:
  PARENT_VERSION: "inconnu"
  PARENT_PIPELINE_ID: "inconnu"
```

Ces variables ont des **valeurs par défaut** ("inconnu"). Quand le pipeline parent déclenche l'enfant, il passe ses propres valeurs qui **écrasent** les valeurs par défaut.

Le job validate-deployment

```
validate-deployment:
  stage: validate
  script:
    - 'echo "Version recue du parent : $PARENT_VERSION"'
    - 'echo "Pipeline parent ID : $PARENT_PIPELINE_ID"'
```

Ce job affiche les variables reçues du parent. C'est la preuve que la communication entre pipelines fonctionne.

Etape 8 — Push et observer le pipeline multi-projets

8.1 — Initialiser le depot

```
git init
git remote add origin https://gitlab.com/VOTRE-UTILISATEUR/tp5-multi-projets.git
```

8.2 — Premier push

```
git add .
git commit -m "feat: pipeline multi-projets avec includes et child pipeline"
git push -u origin main
```

8.3 — Observer dans GitLab

1. Allez sur votre projet > **Build** > **Pipelines**
2. Vous devez voir le pipeline principal avec ses stages
3. Cliquez sur le job trigger-child : vous verrez un **lien** vers le pipeline enfant
4. Cliquez sur ce lien pour voir le pipeline enfant avec ses propres jobs

Ce que vous devez observer

```
Pipeline #1 (parent) :
install-deps    --> OK (installe les dependances)
test-unit       --> OK (lance les tests)
build-app       --> OK (construit l'application)
trigger-child   --> OK (lance le pipeline enfant)
|
+--> Pipeline #2 (enfant) :
      validate-deployment --> OK (affiche les variables du parent)
      notify-team         --> OK (simule une notification)
      deploy-staging      --> OK (deploie en staging)
      deploy-production   --> EN ATTENTE (bouton play, déploiement manuel)
```

Etape 9 — Comprendre le passage de variables entre pipelines

Comment ca fonctionne

Le pipeline parent peut passer des variables au pipeline enfant de deux manieres :

Methode 1 : Dans le bloc trigger (recommandee)

```
trigger-child:
  stage: trigger
  trigger:
    include: child-pipeline.yml
  variables:
    MA_VARIABLE: "ma-valeur"
    AUTRE_VARIABLE: $CI_COMMIT_SHA
```

Les variables definies dans le bloc `variables` du job `trigger` sont transmises au pipeline enfant.

Methode 2 : Variables predefinies CI

Certaines variables CI sont automatiquement disponibles dans le pipeline enfant :

Variable	Disponible dans l'enfant ?	Description
CI_COMMIT_SHA	Oui	Hash du commit
CI_COMMIT_BRANCH	Oui	Nom de la branche
CI_PROJECT_DIR	Oui	Dossier du projet
CI_PIPELINE_ID	Non (c'est celui de l'enfant)	Il faut le passer explicitement

Attention aux variables protegees

Si vous avez defini des variables **protegees** dans Settings > CI/CD > Variables, elles ne sont PAS automatiquement transmises aux pipelines enfants. Il faut les passer explicitement via le bloc `variables` du job `trigger`.

Etape 10 — Recapitulatif

Tableau des concepts

Concept	Mot-cle	Ce que ca fait
Template partage	<code>.nom-du-job + extends</code>	Factoriser la config commune
Include local	<code>include: local</code>	Decouper le pipeline en fichiers
Include file	<code>include: file</code>	Partager entre projets GitLab
Include template	<code>include: template</code>	Templates officiels GitLab
Include remote	<code>include: remote</code>	Importer depuis une URL
Pipeline enfant	<code>trigger: include</code>	Lancer un sous-pipeline dans le meme projet

Pipeline downstream	trigger: project	Lancer un pipeline dans un autre projet
Passage de variables	variables dans trigger	Communiquer entre pipelines
Strategy depend	strategy: depend	Attendre que l'enfant finisse

Arborescence finale du projet

```

tp5-work/
.gitlab-ci.yml          --> Pipeline principal (orchestrateur)
.gitlab/
  ci/
    templates.yml       --> Templates partagés (jobs cachés)
    deploy.yml          --> Jobs de déploiement
  child-pipeline.yml    --> Pipeline enfant (déclenché par le parent)
src/
  app.js
  server.js
tests/
  app.test.js
package.json
.gitignore

```

Schema mental

```

.gitlab-ci.yml (chef d'orchestre)
|
|-- include: templates.yml  --> Fournit .base-node (image + cache + before_script)
|-- include: deploy.yml    --> Fournit deploy-staging et deploy-production
|
|-- install-deps           --> extends: .base-node
|-- test-unit              --> extends: .base-node
|-- build-app              --> extends: .base-node
|
|-- trigger-child -----> child-pipeline.yml (pipeline enfant)
|                           |-- validate-deployment
|                           |-- notify-team
|
|-- deploy-staging         --> (depuis deploy.yml, automatique)
|-- deploy-production      --> (depuis deploy.yml, manuel)

```

Erreurs courantes et solutions

Erreur 1 : "file not found" sur un include local

```
Error: Local file '.gitlab/ci/templates.yml' does not exist!
```

Cause : Le chemin du fichier est incorrect ou le fichier n'a pas été commité.

Solution : Vérifiez que le fichier existe ET qu'il a été ajouté avec `git add`.

```
# Verifier que le fichier existe
ls .gitlab/ci/templates.yml

# Verifier qu'il est suivi par Git
git status
```

Erreur 2 : "jobs config should contain at least one visible job"

Cause : Votre fichier YAML ne contient que des jobs caches (commencant par .). GitLab a besoin d'au moins un job visible.

Solution : Ajoutez au moins un job sans . dans votre pipeline principal.

Erreur 3 : Le pipeline enfant ne recoit pas les variables

Cause : Les variables du parent ne sont pas automatiquement heritees.

Solution : Passez-les explicitement dans le bloc variables du job trigger.

```
trigger-child:
  trigger:
    include: child-pipeline.yml
  variables:
    MA_VARIABLE: $MA_VARIABLE_PARENT
```

Erreur 4 : Script YAML avec ": " casse le parsing

```
Error: mapping values are not allowed in this context
```

Cause : Une ligne de script contient : (deux-points + espace) qui est interprete comme du YAML.

Solution : Toujours entourer ces lignes avec des guillemets simples.

```
# MAUVAIS (erreur YAML) :
script:
  - echo Version : 1.0.0

# BON :
script:
  - 'echo "Version : 1.0.0"'
```

Erreur 5 : "Circular dependency detected"

Cause : Deux projets se declenchent mutuellement (A trigger B, et B trigger A).

Solution : Reorganisez vos triggers pour avoir une hierarchie claire (un seul sens de declenchement).

Pour aller plus loin

- [Documentation officielle : Include](<https://docs.gitlab.com/ee/ci/yaml/includes.html>)

- [Documentation officielle : Multi-project pipelines](https://docs.gitlab.com/ee/ci/pipelines/downstream_pipelines.html)
- [Documentation officielle : Parent-child pipelines](https://docs.gitlab.com/ee/ci/pipelines/downstream_pipelines.html#parent-child-pipelines)

Bravo ! Vous savez maintenant organiser vos pipelines de maniere professionnelle. Dans le TP6, nous verrons les **pipelines dynamiques** qui generent leur configuration a la volee.